CH 02: Probability Distributions

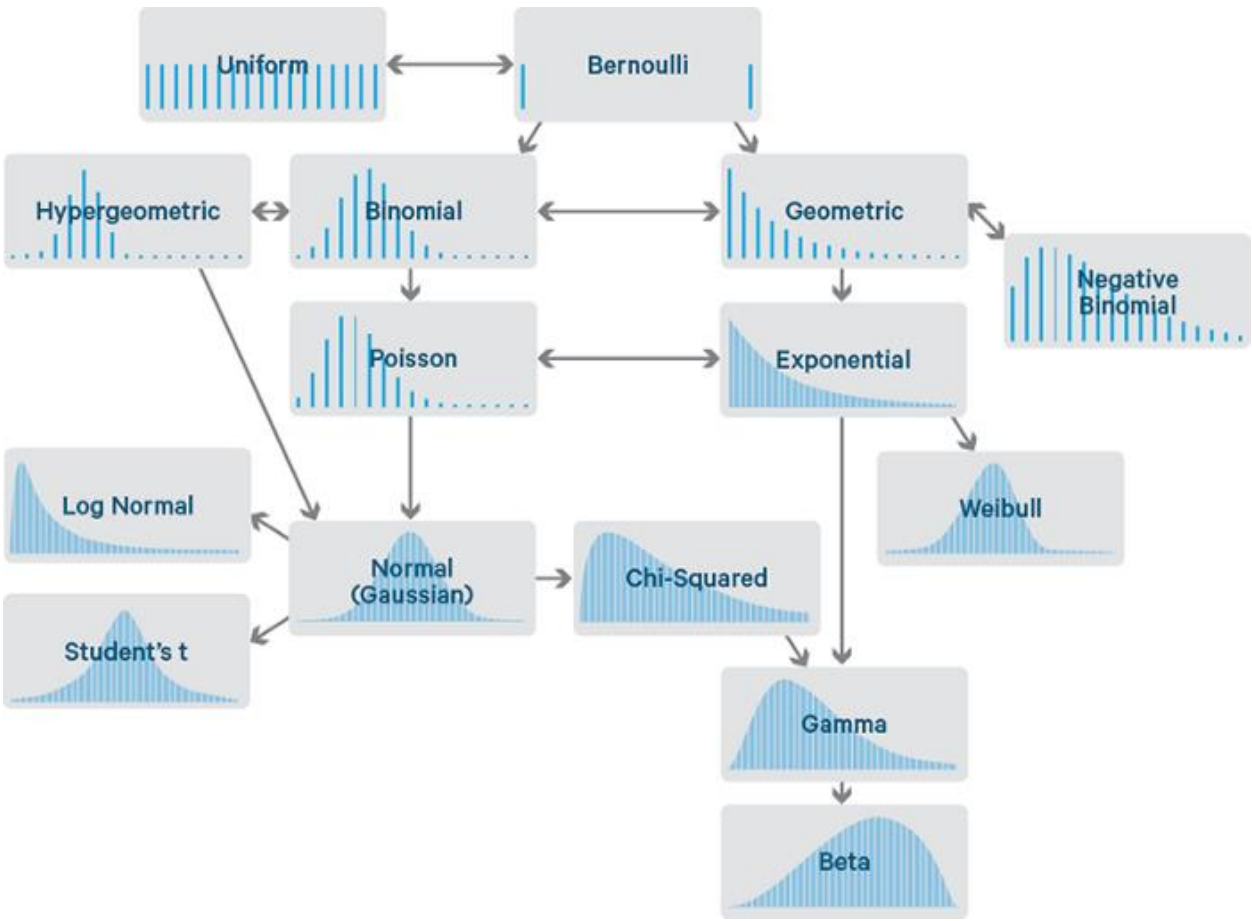
Contents

Introduction 2

Binary Variables 4

Introduction

- This chapter discusses examples of probability distributions.
- A probability distribution is the function that gives the probabilities of occurrence of different possible outcomes for an experiment.
- There are many probability distributions, each has its own application and properties.
  [https://en.wikipedia.org/wiki/List_of_probability_distributions]
- Probability distributions can be:
  - Discrete probability distributions for discrete data.
  - Continuous probability distributions for continuous data.



- The expected value of a function $f(x)$, is the average value under the probability distribution $p(x)$.
    - For a discrete distribution:

$$\mathbb{E}[f] = \sum_x p(x) f(x)$$

    - For a continuous distribution:

$$\mathbb{E}[f] = \int p(x) f(x) \, \mathrm{d}x.$$

- The variance of a function $f(x)$, is the measure of how much variability there is in $f(x)$ around its mean value $E[f(x)]$.

$$\mathrm{var}[f] = \mathbb{E}\left[(f(x) - \mathbb{E}[f(x)])^2\right]$$

Binary Variables

- Consider a random variable $x \in \{0,1\}$. The probability of $x = 1$ depends on a parameter μ:

  $$p(x = 1|\mu) = \mu$$

  such that $0 \leq \mu \leq 1$.

  It follows that:

  $$p(x = 0|\mu) = 1 - \mu$$

  This is known as Bernoulli distribution:

  $$Bern(x|\mu) = \mu^x(1-\mu)^{1-x}$$

- The Bernoulli distribution is normalized.
  1. Normalization is changing the values of numeric columns in the dataset to use a common scale, for example the values are between 0 and 1.
- The Bernoulli distribution have mean and variance given by:

$$\begin{aligned} \mathbb{E}[x] &= \mu \\ \mathrm{var}[x] &= \mu(1-\mu). \end{aligned}$$

- Suppose we have a dataset $D = \{x_1, x_2, \dots, x_N\}$, assuming the data values are drawn independently from function $p(x|\mu)$:

$$p(\mathcal{D}|\mu) = \prod_{n=1}^{N} p(x_n|\mu) = \prod_{n=1}^{N} \mu^{x_n}(1-\mu)^{1-x_n}.$$

- We need to estimate the value of μ to enable us to compute the likelihood of a given variable.
    1. Compute the log of $P(D|\mu)$

$$\ln p(\mathcal{D}|\mu) = \sum_{n=1}^{N} \ln p(x_n|\mu) = \sum_{n=1}^{N} \{x_n \ln \mu + (1 - x_n) \ln(1 - \mu)\}.$$

    2. Set the derivative of $\ln p(D|\mu)$ with respect to $\mu = 0$, we obtain the maximum likelihood estimator (sample mean)

$$\mu_{\mathrm{ML}} = \frac{1}{N} \sum_{n=1}^{N} x_n = \frac{m}{N}$$

where N is the total number of elements, and m is the number of occurrences of an event.

- Import the libraires

```
#%%
import numpy as np
import matplotlib.pyplot as plt

from prml.rv import (Bernoulli,
                     Beta)

np.random.seed(1234)
```

- Create a dataset that has [0, 1, 1, 1].
- Fit a Bernoulli model to the data and print the estimated μ.

```
#%%
data = np.array([0, 1, 1, 1])
model = Bernoulli()
model.fit(data)
print("The estimated mu is: ",
      model.mu)
```

- To check that the model is accurate, substitute the value of μ for $x = 0$ and $x = 1$ in the equation:

$$Bern(x|\mu) = \mu^x(1-\mu)^{1-x}$$

  - For $x = 0$, we get 0.25. For $x = 1$, we get 0.75

- Now suppose we flip a coin, say, 3 times and happen to observe 3 heads ($x = 1$).
  - Then $N = m = 3$ and $\mu_{ML} = 1$.
  - In this case, the maximum likelihood result would predict that all future observations should give heads. (This is overfitting)
- The above experiment can be described as binomial distribution.
  - So instead of

$$\mu^x(1-\mu)^{1-x}$$

    It become

$$\mu^m(1-\mu)^{N-m}$$

  - To maintain normalization, we define the binomial distribution as

$$\mathrm{Bin}(m|N,\mu) = \binom{N}{m}\mu^m(1-\mu)^{N-m}$$

    where $\binom{N}{m}$ means choose m objects of N objects

$$\binom{N}{m} \equiv \frac{N!}{(N-m)!m!}$$

  - The mean and variance become

$$\mathbb{E}[m] \equiv \sum_{m=0}^{N} m\mathrm{Bin}(m|N,\mu) = N\mu$$

$$\mathrm{var}[m] \equiv \sum_{m=0}^{N} (m-\mathbb{E}[m])^2\,\mathrm{Bin}(m|N,\mu) = N\mu(1-\mu).$$

- The function that estimates the value of μ in Bernoulli and binomial distribution is given by the fraction of the observations in the data set having $x = 1$.
    - This can cause overfitting.
    - Try running this code several times.

```
#%%
for i in range(3):
    print("Experiment", i,":",
          model.draw(1))

```

- To solve this problem, we will use Beta distribution. (Discussed next week)